Structuring the empirical chapters

A skeleton for the data-collection / results and discussion / evaluation sections, with the figure and number for each claim already attached. Figures are named by their folder in `docs/report_figures/`: **bench NN** is `01-benchmark/NN-…`, **net NN** is `02-network-sim/NN-…`, and **method 01** is `03-methodology/01-…`. Renumber them for the report once you know which ones you are keeping.

The two chapters can be combined — the structure below works either as two chapters or as one chapter with results and interpretation interleaved per axis. **If you combine them, do it per axis** (tolerance: result then interpretation; cost: result then interpretation), not by putting all results first and all discussion second. Mixed-order combining is where these chapters usually become hard to follow.

Chapter A — Experimental method and data collection

The job of this chapter is to make the results *believable before they are read*. Everything here is about controls.

A.1 What is held constant, and why it has to be

The central methodological claim: comparing erasure codes is meaningless unless storage overhead is fixed, because any code tolerates more erasure if allowed more space. Every scheme runs at **n ≈ 8192, k = 4096, ~2× overhead** (2D-RS at 2.06×, within 3%).

State plainly that an earlier iteration had 2D-RS at 4× overhead, and that comparing a 4×-overhead code against 2×-overhead codes at equal erasure rate flattered it. Describing a correction you made is a strength, not an admission — it shows the control was deliberate.

A.2 The fairness controls

Four, in increasing order of strictness:

1. **One field.** Real BLS12-381 scalar field, not a toy field.
2. **One blob.** Real application bytes packed once via EIP-4844 rules; every scheme encodes that same blob, not seven random ones.
3. **Identical erasure positions.** Where codeword lengths match — 1D-RS and all five BC rows are n = 8192 — trials corrupt the *same positions*, not merely the same rate. 2D-RS has n = 8448 and so draws its own masks at the same rate. Say this explicitly; it is the strictest control available and a reader will wonder.
4. **Success is measured in bytes.** A trial counts only if the recovery round-trips to the exact original application bytes.

A.3 Why byte-level verification, not the decoder's own flag

This deserves its own subsection, and **method 01**. An earlier decoder returned zeros for values it had not solved and reported success: across 40 trials it claimed success every time and was wrong on 31. Had the harness trusted the `remaining` flag, it would have reported 100% success at every erasure rate.

This is the strongest methodological point in the project. It shows the validation caught a fault that would have invalidated every number, and it justifies the expense of byte-level checking. Across the final sweep's

1,800 trials, silent corruption is zero for every scheme.

A.4 Trial allocation and the threshold definition

- Trial counts vary by scheme and rate — 12–20 per rate on a fine grid across each scheme's own transition band, one or two well away from it, where the outcome is certain either way. Give the rule, and note that 95% Wilson intervals are in the JSON.
- **Define the threshold explicitly**: the erasure probability ε at which a scheme recovers half the time, on the binary erasure channel BEC(ε). It is estimated by a maximum-likelihood logistic fit to every trial in the sweep, with a 95% bootstrap interval. Every tolerance number depends on this, so state it once, early, and do not change it.
- **Say why the sampling is dense — it is a strength, not an admission.** An earlier sweep spread its trials evenly and ran as few as three near a threshold, and reading thresholds off it gave 1D-RS = 0.505, above what an MDS code with $n = 2k$ can sustain. The decoder was right (every trial obeyed the $n - k$ rule); three trials just cannot place a threshold to three decimals. The final sweep samples each transition densely, and the exact MDS check shows the method is sound: 1D-RS measures 0.499 (CI 0.497–0.501) against an exact 0.50006, and all 132 of its trials obey the $n - k$ rule. Catching and correcting this is exactly what a careful methodology section should show.

A.5 Reproducibility

Measurement writes JSON; every figure and table is computed from that JSON rather than transcribed. Figure generation is deterministic. State the pipeline — `benchmark_1D_2D_BC.py` → `benchmark_results.json` → `make_figures.py` — and that 1,800 trials over 55 erasure rates underlie the sweep.

Criteria coverage — check this before you write

The research question names six axes, and the Phase 3 brief names three things the network layer must capture. Which figure answers which:

criterion	coding benchmark	network simulation
Fault tolerance	bench `01` (or `02`), `03`, `04`	net `01`, `02`
Computational complexity	bench `05`, `06`	net `07`
Encoding / decoding efficiency	bench `05`, `07`	net `06`, `07`
Storage overhead	held fixed at ~2× — stated in method, not a figure	—
Bandwidth consumption	bench `09` (data needed)	net `04` (data fetched)
Scalability	—	net `08` (peer count)

Phase 3 brief asks for	covered by
Bandwidth saturation	net `04`, at 25 Mbps with six blobs
Packet propagation latency	net `03` (gather time against withholding)
Block processing throughput	net `05` (blobs per slot), `06` (fits in a slot)
Varying adversarial conditions	every network figure sweeps it
Standard network architectures (plural)	**not covered — one topology only**

Two things worth noticing in that table. **Storage overhead has no figure** because it is a controlled variable, not a result — say so explicitly in §A.1 rather than leaving a reader to wonder where it went. And **bandwidth appears twice, but not independently**: the benchmark computes what a scheme needs from its measured threshold, and the simulation sets each node's download target from the same measured tolerance, so the two share an ordering by construction. What the simulation adds is the real overhead above that target — over-request, requests already in flight, re-requests after timeouts. Say that in one sentence in §C rather than presenting it as two methods agreeing.

Chapter B — Results

One section per axis from the research question. Report numbers; save interpretation for Chapter C (or the paired subsection, if combining).

B.1 Fault tolerance — bench 01 (or 02), bench 03

scheme	threshold	95% CI
1D-RS	0.499	0.497–0.501
2D-RS	0.400	0.397–0.403
BC ω512 λ2	0.349	0.347–0.351
BC ω1024 λ2	0.342	0.340–0.343
BC ω2048 λ2	0.336	0.335–0.338
BC ω1024 λ3	0.255	0.252–0.257
BC ω1024 λ4	0.204	0.202–0.206

Bench 01 and bench 02 are the same data. Use one: 01 shows the shape of each transition, 02 is the compact single-column version.

Two findings:

- **1D-RS reaches its exact theoretical value.** 0.499 measured, with the exact 0.50006 for an MDS code with $n = 2k$ inside the interval — what being MDS means, and the check that the harness is sound. Its measured points sit on the exact curve in bench 01(a).
- **λ sets the block-circulant threshold; μ refines it.** Holding $\mu = 4$, raising λ from 2 to 3 to 4 takes it from 0.342 to 0.255 to 0.204, against predictions of 0.333, 0.250, 0.200. Every measurement sits just *above* its prediction — the prediction is one block's repair limit, and the cascade across blocks can only add to it. At fixed $\lambda = 2$ the three rows rise slightly with μ (0.336, 0.342, 0.349 for $\mu = 2, 4, 8$), a real

effect since the intervals do not overlap, with the same cause. Bench 03 is theory and experiment agreeing — lead with it.

- **Do not write that μ "does not matter".** An earlier draft said the λ=2 rows land at the same threshold. At the sweep's resolution they do not; they differ by just over a point, ordered by μ. The honest claim is that λ dominates and μ is second-order.

B.2 Computational cost — bench 05, bench 07

Report encode and decode separately; they are not interchangeable. Decode at 20% erasure: 1D-RS 20.5 s, 2D-RS 2.5 s, BC rows 1.5–5.4 s. Encode runs the other way: 1D-RS 0.23 s, BC ω2048 15.8 s.

Note that all timings are single-threaded Python over `galois`, so they are **relative** measurements, not production figures. Say so once here rather than defending it later.

B.3 Bandwidth — bench 09

Surviving data needed to rebuild one blob, as a multiple of blob size: 1D-RS 1.00× (the floor — nothing rebuilds k symbols from fewer than k), everything else 1.24–1.59×. An earlier figure of 0.99× was below the floor, and was the same threshold error propagating.

B.4 Network behaviour — net 01–07

The simulation's distinguishing feature: in the benchmark the erasure mask is an *input you invent*; here it is an *output* — whatever failed to arrive before the deadline. Make that the opening sentence of the section.

- Realised erasure ≈ p^4 under custody replication 4 (net `01`), the same for all six schemes (2D-RS, with 132 cells rather than 128, differs only in the fourth decimal). This is the network layer's own measurement and does not depend on the coding benchmark.
- **What the thresholds mean in withholding terms** (net `02`): BC ω1024 λ4 breaks first, at 70% withholding (5/30 recovered), and 1D-RS never does up to 80%. The order itself is inherited from the benchmark — the simulation reads recovery off the benchmark's measured curve — so present this as what the thresholds mean operationally, not as an independent confirmation of them. **Do not call it cross-validation**; a marker who reads the method will see why.
- **Latency** (net `03`): withholding costs time, not just data, because the client must burn its request timeout before falling back. The scheme needing the most cells hits the deadline first — BC ω1024 λ4 at 60% withholding — while 1D-RS never reaches it in the sweep.
- **Bandwidth** (net `04`): 1.02 MiB fetched by 1D-RS against 1.54 MiB by BC ω1024 λ4 (the whole codeword), for 0.73 MiB of payload. Same ordering as the benchmark's surviving-data figure by construction; the measured part is the overhead above it.
- **Throughput** (net `05`): blobs reconstructed per slot broadly inverts the tolerance ordering — the scheme surviving the most erasure rebuilds the fewest blobs (1D-RS, 0.39 of six). The one exception is ω2048 λ2, whose large blocks make it slower to decode than 2D-RS.
- **Does it fit in a slot?** (net `06`): at six blobs and no adversaries only BC ω1024 λ3 (5.9 s) and λ4 (1.2 s) finish inside twelve seconds, with ω1024 λ2 just over at 13.7 s; by 40% withholding only λ4 still averages under twelve, and it recovers in 19 of 30. 1D-RS takes 187 s — roughly fifteen slots of work for one slot of data.

B.5 Scalability — **net 08**

Realised erasure against peer count, against the exact hypergeometric probability computed from the model rather than fitted. Measurement lies on prediction. Conclusion: what protects a cell is the *fraction* of the network custodying it, so peer count cannot be varied without specifying replication.

Chapter C — Discussion and evaluation

C.1 Guarantees versus behaviour — **bench 04**

2D-RS tolerates 0.400 against a worst-case guarantee of 0.098 — four times its own bound. Not a contradiction: a minimum-distance guarantee holds against *every* pattern including the adversarial worst case, while these trials draw patterns at random.

This distinction is worth a paragraph of its own, because it is the most likely place for a reader to think they have found an error.

C.2 Why the costs differ — **bench 06**

Raw decode time rises with erasure rate for everything, which mostly says there is more to repair. Divide it out and the structure appears: 1D-RS near 11–13 ms per erasure over most of the range (cost linear in erasures, one long division's worth each); BC rows at 0.85–3.7 ms; 2D-RS falling from 8 ms to 1 ms, revealing a large fixed cost that amortises — which is why it looks expensive at 2% erasure and cheap at 30%.

C.3 The trade-off — **bench 08**

The synthesis. Only three of seven schemes are on the cost-versus-tolerance frontier. BC ω512 λ2 is the cheapest scheme still surviving a third of the codeword; 2D-RS buys ∼5 points more tolerance for about 60% more decode time; 1D-RS buys the maximum and pays an order of magnitude. (Costs are at 21% erasure, the highest rate every scheme still recovers.)

Call out explicitly that **BC ω1024 λ4 is dominated on both axes** by BC ω512 λ2 — a parameter choice with nothing to recommend it. Negative results about your own parameter space are credible and worth stating.

C.4 The asymmetry that changes the recommendation — **bench 07**

Encode is paid once by the proposer; decode is paid by every reconstructing node. 1D-RS encodes in 0.23 s and decodes in 21 s; BC ω2048 spends 15.8 s encoding to decode in 5.8 s (decode at 21%). For data-availability sampling the second shape is the more useful one — an argument for block-circulant the tolerance numbers alone do not make.

C.5 The result that reframes the rest — **net 07, net 06**

Every scheme gathers in about a second; decode runs from about 5 s to 186 s above that, and at six blobs most schemes miss the 12-second slot. The one exception, BC ω1024 λ4, avoids decoding by ending up with the whole codeword. **Bandwidth was the expected constraint and is not close.** Decode arithmetic is the binding one: cutting the link from 25 to 5 Mbps moves 2D-RS's six-blob total by only 6%.

This has a consequence worth stating: it reorders the research question's own priorities. A scheme's erasure tolerance matters less than expected, because custody replication absorbs most peer failure (§B.4) and decode speed dominates the time budget.

C.6 Threats to validity

Be first to raise these. A reader who finds them unprompted discounts everything else.

- **Random, not adversarial, erasures.** Patterns are drawn i.i.d.; a real adversary chooses the worst pattern. Measured tolerances are therefore upper bounds on adversarial performance — directly relevant to 2D-RS and the $\lambda=2$ rows, which exceed their worst-case guarantees.
- **Single-threaded Python.** Relative comparisons hold; absolute seconds would not survive an optimised implementation.
- **The block-circulant rows carry a ~2.3× implementation penalty** that the Reed-Solomon rows do not. Defining the BC local code over roots of unity (to make it FFT-capable) means solving over 255-bit field elements instead of small integers, and that cost is paid even with the transform switched off. Measured in `decoder_version_results.json`. This narrows the apparent 1D-RS gap and moves the 2D-RS crossover; it does not touch any tolerance result. Raise it yourself in §B.2 — it is the kind of thing a reader finds and then distrusts everything else.
- **One blob of application data.** Erasure patterns differ per trial, but the payload is fixed.
- **Block-circulant is measured single-phase — exactly.** Phase 2 implements the paper's auxiliary construction, but that needs the paper's evaluation-point layout and the benchmark uses a shared one for comparability. Replaying all 1,396 BC trials with the decoder's counters on showed phase 2 never ran: in 674 trials a stalled pair was over the 256-unknown budget and was skipped. So BC thresholds are exactly what phase 1 achieves — a limit of this implementation and its layout choice, not of the code. Say which.
- **Network recovery is calibrated, not re-decoded.** The simulation reads recovery and download size off the benchmark's measured curves, so its failure and bandwidth orderings are inherited from the benchmark. `--real-decode` checked decode cost and recovery under clustered erasure, but only at 1–10% erasure, far below every threshold.
- **One network topology**, where the brief asks for several.
- **Simulated network.** Latency and bandwidth are modelled, not measured on real hardware.

C.7 Limitations and further work

Four concrete items, each with a named fix:

1. Implement Algorithm 1's auxiliary GRS construction for phase 2 — would raise the correction radius from ρ toward 2ρ and drop the cost from cubic to quadratic.
2. Replace the NTT path's polynomial division with Newton iteration — the transform is only 12% of that decode; ~80% is a schoolbook division.
3. Run the network's near-threshold points with `--real-decode`, which would turn the inherited failure ordering into an independent check of how the codes behave under clustered, cell-level erasure.
4. Add further network topologies.

Two things to settle before writing

The NTT decoder's status — settled. The roots-of-unity version stays; nothing is being reverted and no re-run is needed. The decode numbers in the report are the ones to write about. The ~2.3× penalty this carries for the block-circulant rows is measured, stated in the report, and listed under threats to validity above — handle it there rather than by adjusting numbers.

Where the NTT investigation lives. It is a genuine negative result — implemented, measured, 7–23× slower, with the cause identified — but it is about *implementation*, not about the coding schemes. It belongs in an implementation appendix or a short subsection of C.7, not in the empirical results. Its four figures are in `docs/figures/` and deliberately left out of `docs/report_figures/`.

Figure budget

There are 18 figures in `docs/report_figures/`, two of which (bench 01 and 02) are alternatives, so 17 in practice — a lot for one chapter. The middle column is there because each figure in it is the only one answering a criterion or a brief item; cut one and that item goes unanswered.

keep (the argument)	keep (criteria coverage)	if space
bench 08 (the trade-off — if you keep one, keep this)	bench 09 (bandwidth)	bench 04 (guarantee vs behaviour)
bench 01 or 02 (tolerance)	net 03 (latency)	bench 06 (cost per erasure)
bench 03 (theory agreeing)	net 04 (bandwidth saturation)	bench 07 (encode vs decode)
bench 05 (decode cost)	net 05 (throughput)	net 02 (withholding at which each breaks)
net 07, net 06 (the bottleneck)	net 08 (scalability)	
net 01 (p^4)		
method 01 (move to methodology)		